Un moteur d'inférence pour eForth

version 1.0 - 17. octobre 2026



Marc PETREMANN

Table des matières

Introduction **3**

L'environnement de développement **4**

Le fichier INFERENCE.FS 4

Le projet main.fs 4

Les autres fichiers 5

Définir des faits **6**

Créer un fait 6

Autodocumenter un fait 7

Création et gestion du vocabulaire facts 8

Listage des faits 8

Remise à zéro des faits 9

Affectation d'une valeur à un fait 9

Définir des règles **11**

Introduction

Un **moteur d'inférence** est le composant central d'un système expert ou d'une intelligence artificielle symbolique, responsable de la prise de décision et du raisonnement automatisé. Il fonctionne en combinant une **base de connaissances** (constituée de faits et de règles de production) avec des algorithmes de déduction pour en déduire de nouvelles informations.

Il utilise principalement deux stratégies de raisonnement :

- **Le chaînage avant**, qui part des faits disponibles pour arriver à une conclusion.
- **Le chaînage arrière**, qui part d'une hypothèse cible pour remonter aux faits nécessaires qui la valident.

En somme, il simule la logique humaine pour résoudre des problèmes complexes sans nécessiter de programmation procédurale rigide.

L’environnement de développement

Pour le moteur d’inférence, nous utiliserons la version eForth Windows. C’est une version FORTH utilisable pour Windows, disponible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__program

La documentation de cette version est accessible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__documentation

eForth ne nécessite aucune installation.

Téléchargez simplement le fichier **uEf64-7.0.7.21.exe** dans un répertoire, à l’emplacement de votre choix, nommé par exemple **eForth**.

Ouvrez ce répertoire **eForth**. Créez un nouveau dossier **INFERENCE**. Téléchargez les fichiers nécessaires depuis le dépôt logiciel. Vous devriez avoir :

```
📁 eForth
    uEf64-7.0.7.21.exe
    INFERENCE.fs
    📁 INFERENCE
        config.fs
        facts.fs
        inference-engine.fs
        main.fs
        rules.fs
        tests.fs
```

Le fichier INFERENCE.FS

C’est le fichier qui permet de se connecter au reste du projet. Contenu :

```
\ load INFERENCE
s" INFERENCE/main.fs" included
```

A chaque fois que vous voudrez compiler le code et utiliser le moteur d’inférence, il suffira de lancer **eForth** en cliquant sur le fichier **uEf64-7.0.7.21.exe**. Une fois eForth démarré, tapez :

```
include INFERENCE.fs
```

Le projet main.fs

L’ensemble du projet est compilé depuis **main.fs**. Exemple de contenu :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
```

Vous pouvez rajouter le chargement d’autres fichiers. Par exemple, pour compilet **tests.fs** :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
include tests.fs
```

Les autres fichiers

Ils sont chargés depuis **main.fs**. Le projet est découpé en fonctionnalités utilisables selon l'évolution du projet. La liste et le contenu des fichiers sources peut évoluer sans préavis et ne plus être ceux décrits dans ce document.

L'ordre de chargement des fichiers est essentiel. Ainsi, on ne peut pas charger le contenu de **facts.fs** avant d'avoir traité le contenu de **inference-engine.fs**.

Définir des faits

Vous pensez maîtriser la logique ? C'est parfait. Alors vous savez manipuler les opérateurs logiques `ET`, `OU`, `NON`… Alors voyons un cas pratique avec un exemple tout à fait trivial :

- la température est très froide
- il pleut

Nous avons deux faits. On peut déduire que s'il pleut et qu'il fait très froid, le risque de gel est réel.

On peut coder ceci en FORTH :

```
 0 constant FALSE
-1 constant TRUE

VARIABLE fait-pluie
VARIABLE fait-froid
VARIABLE fait-gel

: RAZ-FAITS ( -- )
    FALSE fait-pluie !
    FALSE fait-froid !
    FALSE  fait-gel ! ;
```

Nos deux variables `fait-pluie` et `fait-gel` sont les drapeaux à tester, avec les valeurs 0 pour faux, -1 pour vrai. Sur ces deux faits, nous pouvons appliquer ces règles :

```
\ Définition des règles comme des mots Forth standards
: cond-regle1 ( -- flag )
    fait-pluie @  fait-froid @  AND ;
: act-regle1  ( -- )
    cond-regle1 fait-gel ! ;
```

A ce stade, nous ne pouvons pas encore parler d'inférence. Et une seule règle, ici `act-regle1` ne justifie pas la création d'un moteur d'inférence.

Créer un fait

C'est le premier pas vers un moteur d'inférence. Un fait est une variable qui contient un état logique ou une valeur numérique. Pour définir un fait :

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value
  ;
```

Ce qui nous permet de réécrire les variables ainsi :

```
FALSE FACT: fait-pluie
FALSE FACT: fait-froid
FALSE FACT: fait-gel
```

Le mot `FACT:` crée un fait qui est simplement une valeur. L'exécution d'un fait empile sa valeur :

```
21 FACT: temperature
temperature   \ empile 21
```

Autodocumenter un fait

L'idée est de greffer une chaîne de caractères qui informe sur la nature du contenu du fait déclaré. EForth sait gérer les chaînes terminées par zéro :

```
z" Température extérieure" z>s type  \ affiche le texte :  Température extérieure
```

Pour greffer ce type de chaîne à un fait, on va simplement compiler l'adresse de cette chaîne après la valeur :

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value                   \ create value
    ,                       \ compile pointer to zstring
  ;
```

Et s'utilise comme ceci :

```
z" Température extérieure" 21 FACT: ext-temp
```

L'exécution de `ext-temp` empile la valeur 21.

Pour récupérer la chaîne de caractères compilée après cette valeur, il faut explorer le champ de données de `ext-temp` :

```
' ext-temp >body cell + @ z>s cr type
\ affiche: Température extérieure
```

Pour faciliter l'accès aux données dans le champ de données d'un fait créé par `FACT:` , on définit cette structure :

```
structures
struct __FACTS
    ptr field ->value       \ fact value
    ptr field ->zstring     \ point to zstring
forth
```

Pour connaître le statut d'un fait, on va définir ce mot :

```
\ display zstring and value in fact
\ example: ' <fact> explain-fact
: explain-fact { xtAddr -- }
    xtAddr >name type
    xtAddr >body ->zstring @ z>s cr  4 spaces type ."  = "
    xtAddr >body ->value @ { value }
    value 0= if
        ." FALSE" exit
    then
    value -1 = if
        ." TRUE" exit
    then
    value .
  ;
```

Et s'utilise ainsi :

```
' ext-temp explain-fact   \ display:
\ ext-temp
\     Température = 21
```

Création et gestion du vocabulaire facts

En l'état, tous les faits sont définis dans le vocabulaire `FORTH`. Si on mixe des définitions classiques, des règles, des faits, l'exécution de `vlist` va restituer une liste de mots difficile à gérer. L'idée serait donc de définir tous les faits et seulement ces faits dans un vocabulaire spécifique :

```
vocabulary facts    \ new facts defined in this vocabulary
```

Pour basculer une création de faits dans ce vocabulaire, on définit des sélecteurs de vocabulaires :

```
: in-facts ( -- )
    facts definitions
  ;

: in-forth ( -- )
    only
    forth definitions
  ;
```

Ces sélecteurs sont incorporés à la définition de `FACTS:` :

```
\ create a new fact
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    in-facts
    value                   \ create value
    ,                       \ compile pointer to zstring
    in-forth
  ;
```

Définissons ces faits :

```
z" Pleut-il" FALSE   FACT: fait-pluie
z" Température extérieur" -4   FACT: ext-temp
z" Gèle-t-il" FALSE  FACT: fait-gel
```

Ces faits seront définis dans le vocabulaire `facts` et sont visibles :

```
facts vlist    \ affiche: fait-gel ext-temp fait-pluie
```

Listage des faits

L'idée sous-jacente de gestion des faits dans le vocabulaire `facts`, c'est de permettre ensuite le listage détaillé. Pour ce faire, on définit ce mot :

```
forth definitions also internals
: list-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
    while  dup explain-fact cr cr >link
    repeat
```

```
    drop
    forth
  ;
only
```

L'exécution de `list-facts` sélectionne le vocabulaire `facts` et récupère le code exécutable de chaque fait. Ce code exécutable est exploité par `explain-fact`. Résultat :

```
list-facts    \ display:
\ fait-gel
\     Gèle-t-il = FALSE
\
\ ext-temp
\     Température = -4
\
\ fait-pluie
\     Pleut-il = FALSE
```

Cette liste de faits affiche les faits et sur la ligne suivante l'auto-documentation et la valeur du fait. Le mot `list-facts` permet donc de tracer tous les faits traités par le moteur d'inférence.

Remise à zéro des faits

La manière dont les faits sont listés est exploitée pour remettre à FALSE tous les faits :

```
\ reset fact value to FALSE
: (reset-fact) ( xt -- )
    >body FALSE swap !
  ;

\ reset values of all facts defined in facts vocabulary
forth definitions also internals
: reset-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
    while  dup (reset-fact) >link
    repeat
    drop cr ." All facts set to FALSE"
    forth
  ;
only
```

Le mot `reset-facts` analyse chaque fait déclaré dans le vocabulaire `facts` et remet sa valeur à FALSE.

Affectation d'une valeur à un fait

Dans le fichier **vocForthExtensions.fs** se trouve le mot `val-input` :

```
\ wait a value and push it on stack
: val-input ( -- n )
    PAD 80 ACCEPT
    PAD SWAP
    EVALUATE
  ;
```

Le contenu du fichier **vocForthExtensions.fs** est compilé par **main.fs** avant la compilation du moteur d'inférence. Le mot `val-input` attend la saisie d'une valeur numérique depuis le clavier et empile sa valeur. Utilisation :

```
facts
cr ." Température actuelle: " val-input to ext-temp  \ saisir 27
ext-temp ,   \ affiche 27
```

A noter que vous pouvez également saisir `TRUE` ou `FALSE`, car `val-input` effectue une évaluation de la valeur saisie. Si cette valeur est une constante ou valeur du vocabulaire FORTH, ce mot sera exécuté et sa valeur déposée sur la pile de données.

Définir des règles

Les règles traitent les faits en deux étapes :

1. vérifie un ou plusieurs faits ;
2. traite un ou plusieurs faits si la règle est vérifiée

Exemple :

```
FALSE value F1
FALSE value F2
FALSE value F3
: R1  ( -- )   \ our first rule
    F1 F2 and if
        TRUE to F3
    then ;
```

Rien de bien compliqué. Factorisons la partie *condition* et la partie *action* :

```
: condition-R1 ( -- fl )
    F1 F2 and ;
: action-R1 ( -- )
    TRUE to F3 ;
: R1  ( -- )   \ our first rule
    condition-R1 if
        action-R1
    then ;
```